

Cross-Agent Shared Memory (CASM) System: Design & Implementation Guide

1. Overview of CASM

What is CASM? Cross-Agent Shared Memory (CASM) is a unified, persistent memory layer that multiple AI agents can read from and write to collaboratively. It serves as a “single source of truth” for agent teams, analogous to how a shared document enables real-time collaboration in place of isolated files. In today’s multi-agent AI setups, agents often operate with separate, siloed memories, leading to duplication of work, inconsistent context, and wasted token usage ¹. CASM addresses this by providing a **common memory substrate** where agents collectively store facts, observations, and intermediate results, enabling synchronized decision-making and knowledge sharing ² ³.

Why is CASM needed? Most multi-agent failures stem not from poor communication but from lack of shared memory ¹. Without CASM, one agent may forget or be unaware of what another has learned, causing redundant actions and misaligned states. For example, Agent A might repeat a web search Agent B already performed because there’s no shared memory of that result. Or Agent X might make a decision using outdated assumptions that Agent Y already corrected in a separate memory. CASM prevents such issues by ensuring all agents have access to the latest information and context. It preserves conversation history, world knowledge, and task state beyond any single agent’s context window. As a result, teams of agents can maintain continuity over long tasks and across sessions – much like how humans rely on shared documents or databases to coordinate.

Key use cases:

- *Multi-agent workflows:* In complex tasks (e.g. a legal analysis pipeline), one agent can add findings to shared memory for others to pick up ⁴ ⁵. CASM allows sequential agent handoffs without losing context – each agent sees what prior agents have contributed.
- *Real-time collaboration:* For simultaneously active agents (brainstorming, monitoring, planning in parallel), CASM enables **live updates** so agents react to each other in real-time. This is akin to Google Docs for AI agents, allowing concurrent editing with automatic conflict resolution ⁶ ⁷.
- *Persistent user memory:* CASM can store long-term facts about users or environments (preferences, prior interactions) that all agents consult. For instance, an agent handling scheduling and another handling email could both reference a shared profile of user preferences.
- *Cross-agent learning:* Knowledge discovered by one agent (e.g. new research data) can be written to CASM and later retrieved by another agent solving a related problem. Over time, the shared memory becomes a **collective knowledge base**, reducing repeated queries and improving answers.

In summary, CASM transforms a collection of powerful but isolated agents into a coordinated team with collective intelligence. It provides real-time collaboration, context preservation, and a consistent knowledge store that grows with experience ⁷. By solving the “memory crisis” in multi-agent systems, CASM unlocks higher efficiency and more advanced use cases than what isolated agents can achieve alone.

2. Technical Architecture

A CASM system's architecture has several core components and design considerations. It must reliably store diverse memory items, support fast retrieval (including semantic search), handle concurrent updates from many agents, and maintain consistency via versioning and conflict resolution. Below we break down the technical design:

- **Memory Storage Engine (with CRDTs):** At the heart of CASM is a backend data store to hold the memory **units** (sometimes called memory blocks or entries). Each memory unit could be a piece of text (conversation message, observation, summary), a structured fact, or an embedding vector. A robust choice for storage is a transactional database (SQL or NoSQL) augmented with conflict-free replication. Using Conflict-Free Replicated Data Types (CRDTs) can ensure that distributed updates converge consistently without data loss ⁸. For example, if two agents in different regions update the same memory item, a CRDT-based store will automatically merge changes in a mathematically consistent way ⁹. This avoids classic race conditions. In early stages, you might deploy a single primary database for simplicity; however, designing the data model with eventual consistency in mind (timestamps, last-writer-wins or CRDT merge rules) will future-proof the system for multi-master or offline scenarios. The memory store should support **append-only logging or multi-version concurrency control** so that old versions of memory can be kept (for audit or rollback) while new updates don't block reads.
- **Data Schema and Memory Graph:** It's useful to impose a schema on memory items for structure and queryability. CASM can store data as JSON documents or rows with defined fields (e.g. `type`, `content`, `timestamp`, `agent_id`, `metadata`). Enforcing schemas ensures data sanity and helps agents interpret memory correctly ¹⁰. For instance, you might define one schema for "conversation message" (with fields like `speaker`, `content`, `session_id`) and another for "knowledge entry" (with fields for `subject`, `relationship`, `object`, `validity period`, etc.). Internally, **organizing memory as a graph** yields powerful capabilities: each memory node (an entity or idea) can link to others, forming a knowledge graph. Edges can represent relations ("Report X was created by Agent Y" or "Fact A supports conclusion B"). A *shared memory graph* acts as a communication backbone: agents collectively update and traverse this graph, which holds the evolving state of their world ². Making the graph temporal – i.e. time-stamping nodes and edges – allows the system to maintain a history of changes (what was known when and by whom) ¹¹ ¹². This concept of a **Temporal Knowledge Graph** brings memory closer to how humans remember: knowledge isn't static but evolves over time. It lets CASM answer queries like "what changed about user preferences last week?" or resolve conflicting info by time ¹³. In practice, the graph can be stored in a graph database (e.g. Neo4j) or simulated in a relational/JSON DB via tables for entities and relationships ¹⁴. The graph model ensures that multiple agents can write to the same knowledge space with built-in versioning, timestamps, and provenance metadata on each entry ¹³. This provenance (e.g. which agent added a node and when) is critical for later conflict resolution and audit.
- **Vector Index for Semantic Search:** Not all memory retrieval can be exact-match; agents often need to find relevant info by meaning. CASM incorporates a **vector index** (embedding-based index) to enable semantic lookups. Every textual memory item (e.g. a conversation message or a document summary) can be embedded into a high-dimensional vector space using an embedding model (such as SBERT or OpenAI embeddings). These vectors are stored in a specialized vector database or index structure, allowing similarity search. For example, an agent can query CASM with an embedding of its current question to retrieve semantically similar past knowledge. The system might use an open-source vector DB like **FAISS**, **Milvus**, **Weaviate**, or **Qdrant**, or a database extension (PostgreSQL with pgvector) ¹⁵. A startup-friendly approach is

Postgres + pgvector for simplicity (letting you use SQL and vector search in one system) ¹⁵, or a managed service like Pinecone for scalability. In Eion (an open-source CASM example), they use Postgres with pgvector to store conversation histories and enable semantic similarity queries ¹⁶ ¹⁷. The vector index should be updated whenever new memory is added (e.g. store the embedding along with the content), and exposed via an API for queries like “search_memory(text or embedding) → returns top-K relevant memory items” ¹⁸. This feature implements Retrieval-Augmented Generation (RAG) for agents: agents pull in relevant long-term memories on the fly instead of being limited to short context windows ¹⁹.

- **APIs and Access Layer:** CASM is offered as a service that agents communicate with (likely over HTTP or a gRPC API). The API design should be simple and flexible. Key API endpoints (or RPC methods) typically include: **Add Memory** (write a new memory item or batch of items), **Get Memory** (fetch memory by key or ID, e.g. retrieve conversation history for a session), **Search Memory** (semantic search using vectors or filters), **Update Memory** (modify an existing entry or its metadata), and **Delete Memory** (if needed for cleanup or privacy). For knowledge graphs, additional endpoints like **Link/Unlink Nodes** or **Query Graph** (to find connections between entities) are useful ¹⁸. Eion’s API, for example, provides tools like `add_memory`, `get_memory`, `search_memory` for conversational data and `create_knowledge`, `search_knowledge` for the knowledge graph ¹⁸. Under the hood, these calls translate to database operations (inserts, queries) or graph queries. CASM should also support a **subscription/notification mechanism**: agents may register to be notified (via WebSocket or pub/sub) when certain memory keys update. This enables reactive behavior – e.g. Agent B subscribes to “plan” updates written by Agent A and wakes up when new content appears. The memX project (a lightweight CASM) demonstrates this with a real-time publish/subscribe on keys, so changes propagate instantly to all agents listening ²⁰. Real-time sync ensures that concurrently working agents stay on the same page.

- **Versioning and Conflict Resolution:** In a multi-agent environment, it’s possible for two agents to try to update the same memory near-simultaneously or for one agent’s new info to contradict another’s earlier entry. CASM must handle such conflicts gracefully. A straightforward approach is to implement **optimistic concurrency control** with timestamps or version numbers on memory entries. If two writes conflict, the system could compare timestamps (last-write-wins by default) or, better, attempt an intelligent merge. For instance, if Agent A and B edit different fields of a knowledge record, the system can merge changes; if they edit the same field with different values, the system might keep both as alternate versions or choose one based on a policy (e.g. trust level or latest timestamp). **CRDT data structures** shine here: by designing memory updates as commutative operations (additions to sets, increments to counters, etc.), many conflicts resolve automatically by CRDT rules ⁹. Even for free-form text, operational transforms or CRDTs can merge concurrent edits (much like collaborative editing in Google Docs). CASM should log all changes with a version ID and timestamp (and perhaps the agent ID who made the change). This not only helps with merging but also provides an audit trail. Eion’s system, for example, features *temporal conflict detection with intelligent merging* – essentially detecting when two agents touch the same knowledge in overlapping time frames and auto-merging or queueing resolutions ¹⁶. In practice, implementing full CRDT for complex data can be involved; an MVP might start with simpler rules (like last-writer-wins plus storing the previous version for manual review). As the system matures, you can introduce more sophisticated conflict resolution strategies (e.g. domain-specific merging rules or even an agent that audits and reconciles conflicts). The **memory graph** structure also assists here: since each knowledge entry is time-stamped and has provenance, conflicts can be resolved by referring to history (perhaps an agent changed something that was true in the past but not now, so the system retains both with appropriate validity intervals ²¹ ²²).

- **Security and Access Control:** As a shared infrastructure, CASM must ensure that only authorized agents or users access or modify certain memories. Each agent (or human user session) should have credentials (API key or token) and an associated permission scope. The system can implement role-based access – for example, some agents may have read-only access to certain knowledge domains, others may have full write access. At startup-scale, a simple API key mechanism with scopes defined in a config (mapping keys or prefixes to allowed agents) is effective ²³ ²⁴. Eventually, you might integrate more robust auth (JWTs with claims about accessible memory realms, or integration with an identity service). Eion’s approach is to have agent registration with assigned capabilities (like “memory:read-write” vs “knowledge:read-only”) and enforce that on API calls ²⁵. All access and changes should be logged for monitoring and auditing.

Architecture Example: One concrete architecture uses **PostgreSQL** as the primary data store (storing JSON documents for memories and vectors via pgvector) and **Neo4j** as a complementary graph store for knowledge relations ¹⁶ ¹⁷. A **Go or Python service** sits in front, providing REST/JSON and WebSocket endpoints, handling schema validation, embeddings, and pub-sub notifications. This service uses a CRDT or versioning library to merge updates. For embeddings, it might call a Python library (e.g. SentenceTransformers) or a microservice to generate vectors ²⁶ ²⁷. The system can scale by separating concerns: the vector index could be moved to a dedicated vector DB cluster (like Pinecone, Weaviate) if query load grows, and the knowledge graph could be a separate service if it becomes large.

Example: Agents collaborating via a shared memory (Eion demo). Multiple AI agents write and read from a unified memory store (Postgres + Neo4j), enabling real-time context sharing ⁷ ¹⁶.

In summary, CASM’s architecture marries **database principles (consistency, schemas, queries)** with **modern AI tooling (embeddings, knowledge graphs, CRDTs)** to create a reliable memory layer. The design should start simple (one node, basic APIs) but allow evolution to a distributed, conflict-tolerant system as you scale to more agents and higher throughput.

3. Step-by-Step MVP Build Guide (6–12 Weeks)

Building a CASM system from scratch is an ambitious project. A phased approach over 2–3 months will help deliver a Minimum Viable Product with core functionality, which can then be expanded. Below is a week-by-week (approximate) development plan:

Week 1-2: Requirements and Design

- **Gather requirements & define scope:** Clarify what types of memory your initial agents need (e.g. just chat history and vector search, or also structured knowledge). Decide on the MVP features: for instance, you might start with textual memory + semantic search, and defer the full knowledge graph for later.
- **Architectural design:** Choose the tech stack (see Section 4) and design the data model. For MVP, map out 2–3 core data entities such as `MemoryItem` (to store a piece of content plus metadata) and perhaps `KnowledgeLink` (if implementing relationships). Plan the API endpoints for create/read/search.
- **Select embedding model:** Pick a pre-trained embedding model for text (e.g. all-MiniLM-L6-v2 as used by Eion ²⁶, or OpenAI’s text-embedding-ada for quick start). Ensure you have access to the model or API and understand its vector dimension.
- **Dev environment setup:** Set up repository, continuous integration (if desired), and get baseline infrastructure running (e.g. spin up a Postgres instance or Redis instance, create a skeleton web service using your chosen framework). If using Docker, write a `docker-compose.yml` that can run the service plus any backing DB (similar to Eion’s quickstart with Postgres and Neo4j ²⁸).

Week 3-4: Core Memory CRUD API

- **Implement data storage:** Create the database schema or data structures for memory. For example, in Postgres define a `memories` table (id, content, metadata JSON, embedding vector, timestamp, etc.), and enable the vector extension ²⁹. Alternatively, if using a document store or Redis, set up keys and any necessary indices.
- **Develop the service endpoints:** Implement the **Add Memory** API (e.g. an HTTP POST `/memory` that accepts content and metadata). Upon adding, store the record in the DB and also generate & store its vector embedding. Implement the **Get Memory** API (GET `/memory?query=params` or `/memory/{id}`) to retrieve entries. If conversation sessions are a concept, support a query by session or agent.
- **Basic retrieval:** Implement a simple **Search** API. At this stage, you could use keyword search or filters. If using Postgres, perhaps utilize text search or basic vector similarity if set up. Ensure that the API can return a list of relevant memory items given a query or vector.
- **Test with dummy agents:** Simulate a scenario with a couple of agent-like scripts calling the APIs. For example, Agent A posts a memory ("User asked about X"), Agent B queries it. Verify data flows correctly and persists.
- **Iteration:** Refine data model or API payloads as needed once you see it working. Keep the design stateless (the server doesn't hold session in memory; all state is in the DB) so it's easier to scale and maintain.

Week 5-6: Semantic Search and Vector Index

- **Integrate embedding generation:** Connect the chosen embedding model to your pipeline. If using a local model (via a Python library or Transformers), write a module for it and ensure it's loaded at service start. If using an API (OpenAI, etc.), wrap calls to it. When new memory is added, generate its vector asynchronously if possible (to keep API response quick, you might enqueue embedding computation in a background thread or job). Store embeddings in the vector index. For Postgres/pgvector, this means just inserting into a column of type `vector`; for an external vector DB, use their SDK to upsert the vector with an ID.
- **Advanced search API:** Enhance the **Search Memory** endpoint to perform **semantic similarity search**. For example, if a query comes in as text, the service should embed the query text, then query the vector index for nearest neighbors, and return those memory items. Support combining filters with vector search (e.g. find similar items but only within the same user or topic, if metadata allows). This is crucial for episodic memory retrieval ¹⁹.
- **Evaluate performance:** With a few hundred test entries, verify that search is fast enough (< few hundred ms). If using a local vector library, tune the search parameters (e.g. number of clusters/probes in FAISS or search k in Milvus) for a good trade-off of speed vs accuracy.
- **Implement pagination or limits:** Ensure the API doesn't choke on large results – typically return top k results for searches, and allow clients to request more if needed.
- **Memory compression (optional):** If dealing with potentially long dialogues, consider implementing a simple summarization or rolling compression of very old memory, as described in context window management ³⁰ ³¹. This might be optional in MVP unless immediate need.

Week 7-8: Knowledge Graph & Advanced Memory (Stretch Goal)

This phase can run in parallel or be deferred based on priority.

- **Graph data model:** If including a knowledge graph, implement an initial model. This could be as simple as a table for `entities` and one for `relationships` (triples), or using a graph DB via its client. Develop APIs like `/knowledge` POST (to add a node or relationship) and GET (to query relationships or search nodes). At first, you might not implement a full graph query language, but you can support queries like "find all facts about Entity X" or "find connections between X and Y" by traversing one hop.
- **Extraction logic:** If possible, add a simple pipeline where new memories (e.g. a conversation transcript) are run through an information extraction step (could be a basic NER or regex) to identify

potential knowledge graph entries. For example, if an agent writes “Alice hired Bob in 2021”, the system could automatically extract (Alice - employed -> Bob) with a timestamp. This can populate the knowledge graph without manual input. In Eion, an in-house extraction service does this to build the temporal KG from chat logs ¹⁷. For MVP, you can implement a rudimentary version or insert a few manual entries to demonstrate the idea.

- **Time/version handling:** Introduce a notion of versioning on knowledge entries. For example, if a relationship is updated, keep the old one but mark it outdated (or store an `end_date` on it). This will lay groundwork for temporal queries (“what was the fact at time T”).

- **Test knowledge usage:** Simulate an agent query that requires multi-hop reasoning. E.g., Agent queries CASM: “Who does Alice employ?” and CASM finds Alice->Bob from the knowledge store. While building a full reasoning engine is beyond MVP, ensure the data needed is being captured and can be retrieved with simple logic.

Week 9-10: Concurrency, Conflict Resolution, and Polishing

- **Implement basic conflict handling:** Add safeguards for concurrent writes. For example, use DB transactions or `SELECT ... FOR UPDATE` when updating a memory record to avoid race conditions. Assign each memory entry a version number; require clients to send the last-known version when updating (to detect mid-air collisions). For MVP, a simple last-writer-wins strategy is okay, but log conflicts for manual review. Consider adding a field like `last_modified_by` and `last_modified_at` on entries.

- **CRDT prototype (optional):** If aiming to showcase CRDT, implement one CRDT data type in the system. A common one is a **GCounter** or **PnCounter** (grow-only or increment-decrement counter) to aggregate some metric from agents without conflict. Or a last-writer-wins register for certain fields (this is technically a basic CRDT where the “highest” timestamp wins ⁸). This could be demonstrated for something like an “ongoing summary” that multiple agents append to. Evaluate how you’ll merge list or text data – if not via formal CRDT, maybe by always appending and later summarizing (thus avoiding direct conflict).

- **Security & ACL:** Implement API key verification on each request. Generate a few static keys for testing, and enforce scope: e.g. one key can only call memory endpoints, another can call both memory and knowledge. This ensures from the start that you have a multi-agent/multi-tenant mindset. (In later production, you’d integrate a proper identity service or dynamic key generation, but MVP can use a config file of keys or simple token check.) ²³

- **Monitoring & Logging:** Add basic logging for each API call (who called, what was done). Also instrument simple metrics: e.g. count of memory items, number of writes per minute, etc., to help observe usage. You might integrate an open-source APM (Application Performance Monitoring) or just output to logs that can be parsed. Memory usage and response times should be tracked to catch any performance issues early.

Week 11-12: Testing, Deployment Prep, and Demo

- **Rigorous testing:** By now you have a fair number of features; write unit tests for the service logic (especially merging, search correctness) and integration tests that spin up the whole system and simulate a multi-agent scenario. Test edge cases: what happens if two `add_memory` calls come in at once? Does search return fresh results after an add? Are unauthorized calls blocked? Consider adversarial tests (agent writes ridiculously large content, or injection strings) to ensure robustness.

- **Documentation:** Document the API (perhaps using an OpenAPI/Swagger spec for the HTTP endpoints). This makes it easier for others (or even your future self) to integrate new agents or client apps. Clearly explain the data model and any limitations. If time permits, write a short **developer guide** describing how to register an agent and use the CASM APIs (Eion, for example, provides a developer console and examples for integration ³² ³³).

- **Deployment setup:** Configure the service for a cloud deployment. This might involve creating a Docker image and pushing to a registry, preparing cloud database instances, setting environment

variables for secrets/keys, etc. Ensure that the system can be launched with one or two commands (ease of deployment is key for a startup product).

- **Demo and iterate:** Build a small demo application or script showing CASM in action – e.g. two agents solving a task using shared memory (similar to memX’s demo with QueryAgent, ExplorerAgent, SynthesizerAgent writing to shared context ³⁴). This not only validates the system end-to-end but also provides a compelling demonstration to stakeholders or early users. Gather any feedback and use the remaining time to fix critical bugs or UX issues in the API (maybe the API response format needs tweaking, etc.).

By the end of 12 weeks, the MVP CASM should support a handful of agents (10+ as required) collaborating with a basic shared memory. It won’t be fully optimized or feature-complete, but it will prove the concept and provide a foundation to build upon. Throughout the process, you may not find explicit step-by-step guides in literature (this plan is derived from best practices and analogous projects, rather than a direct source), so continuous testing and adjusting will be crucial. The key is to deliver a working core early (by week ~4-6) and then harden and expand it, rather than trying to build the “perfect” system in one go.

4. Recommended Tech Stack

Choosing the right stack is critical for performance, scalability, and developer productivity. Below we list recommendations for each major component, with rationale:

- **Programming Language:** For speed of development and rich AI/DB libraries, **Python** is a strong choice initially. Python has excellent libraries for machine learning (e.g. Transformers for embeddings), database ORMs/clients, and web frameworks (FastAPI, Flask) to quickly build APIs. In fact, a reference implementation (memX) uses Python + FastAPI for the server ²⁰. However, be mindful of Python’s performance and concurrency limits – for a higher-scale production, you might consider **Go** or **Node.js** for the server, or even **Rust** for maximum performance. Eion, for example, is implemented in Go for the core server (leveraging Go’s concurrency and efficiency), but also uses Python for certain tasks like embedding computation ³⁵ ³⁶. One pragmatic approach: start with Python for the MVP (fast iteration), then refactor latency-critical parts or the entire service into Go once the design solidifies. Go has database drivers and can call out to Python services for ML if needed, and its strong typing can reduce runtime errors as the codebase grows.
- **Backend Database (Memory Store):** A **SQL relational database** is recommended for the persistent memory store because of its reliability and flexibility with structured data. **PostgreSQL** in particular offers JSON storage (JSONB columns for flexible schemas) and the pgvector extension for embeddings, making it a compelling one-stop solution ¹⁵. With Postgres, you get ACID transactions (useful for consistency on concurrent writes) and you can start small (even a single instance or container) and scale to clusters or cloud managed services as needed. Many early-stage AI startups use Postgres with pgvector to avoid adding a separate vector DB initially ³⁷ ³⁸. Alternatively, if your data is mostly unstructured or you prefer NoSQL, **MongoDB** could be used (MongoDB can store JSON documents and has multi-document ACID support in latest versions; it doesn’t natively do vector search, but you could integrate a vector engine or use MongoDB Atlas Vector Search if available). Mongo’s flexible schema might suit rapidly evolving memory structures, and indeed MongoDB has promoted the concept of memory engineering for AI agents ³⁹ ⁴⁰. That said, adding vector support might be less straightforward than Postgres or a purpose-built vector store.

- **Vector Database/Index:** If not using a database that supports vectors, you will need a dedicated vector search solution. For managed services, **Pinecone** or **Weaviate (Cloud)** are popular – they handle scaling and approximate nearest neighbor search out-of-the-box. Pinecone, for instance, is used with frameworks like LangChain to store long-term memory and can scale to billions of embeddings, though it introduces ongoing cost. Open-source options include **FAISS** (which can be embedded in your Python service for small scales), **Qdrant**, **Milvus**, or **Weaviate (self-hosted)**. Weaviate and Milvus offer REST/gRPC APIs and can also store some metadata with vectors. If your use-case involves complex boolean filters plus vector similarity (e.g. “find relevant memory where agent=Finance and topic=Market”), ensure the vector DB supports that (Weaviate does, Pinecone does via metadata filters). Another interesting option is **Redis** with the RediSearch module: Redis now supports vector similarity queries on vector fields, and can store hashes for metadata. Using Redis, you could combine fast in-memory retrieval of vectors with other data and even use Redis’s pub/sub for notifications – effectively covering multiple CASM needs in one system ²⁰. In fact, memX’s design leverages Redis under the hood for real-time updates and state sync (with a FastAPI layer on top) ²⁰. However, for more than trivial amounts of data, a disk-backed or file-backed store (Postgres, Pinecone, etc.) will be safer to prevent data loss and handle larger memory.

- **Knowledge Graph Store:** If you plan to implement a rich knowledge graph, a graph database like **Neo4j** is a strong candidate. Neo4j is ACID and optimized for traversing relationships, and it allows adding attributes (like timestamps) to nodes/edges which is ideal for temporal knowledge graphs. Eion uses Neo4j for exactly this reason, to store temporal knowledge with entities and relationships, connected to their memory system ¹⁶ ¹⁷. Neo4j can be introduced alongside Postgres (they can coexist, as in Eion’s architecture). The downside is operating another database adds complexity; if you want to avoid that at MVP stage, you could simulate a graph in Postgres using tables and foreign keys (or use an embedded graph library in-memory and periodically persist it). Another alternative is **ArangoDB** or **JanusGraph**, multi-model databases that support graph and also other data models. ArangoDB, for instance, supports a graph data model and also has some full-text search and even limited vector similarity capability via extensions, so it could theoretically handle memory docs and graph in one system. However, Arango is less commonly used in the LLM-agent space compared to the Postgres + Neo4j combo. For simple knowledge linking, even a Python library like **NetworkX** could be used to prototype how you’d navigate the knowledge, but you’d eventually need a persistent store for it. We recommend using a proven graph DB if the knowledge graph is central to your memory; otherwise, postpone this component until you truly need complex relational queries.

- **Embedding & ML Libraries:** For generating text embeddings, **Sentence Transformers (via HuggingFace)** is an excellent library in Python. Models like `all-MiniLM-L6-v2` (384-dimensional) are lightweight and give good semantic similarity performance ²⁶ – suitable for a starting point. If you require multilingual support or more accuracy and can afford external API calls, OpenAI’s embedding APIs (like `text-embedding-ada-002`) produce 1536-dimensional vectors of high quality, at a cost. The choice might come down to cost vs performance – self-hosting a smaller model is cheaper and keeps data local, while API models might perform better at a price. Ensure whichever path, the embedding generation is thread-safe or can be scaled (maybe running in its own service to avoid blocking the main API).

For other ML tasks: if doing summarization/compression of memory, you might use HuggingFace transformers for sequence-to-sequence models or simply OpenAI/Anthropic APIs for quick results. Python’s rich ecosystem makes it easy to integrate these. In a Go implementation, you’d likely call out to a Python microservice or use an API, since Go lacks native ML support.

Additionally, if you plan to incorporate any *prompting or LLM calls as part of memory (e.g. an agent*

query triggers CASM to summarize something via LLM), libraries like **LangChain** or **LlamaIndex** can be integrated on the client side. These aren't part of CASM's core stack, but CASM should be compatible with them (for example, LangChain's memory classes could use CASM's API as a backend store).

- **Web Framework / Server:** For Python, **FastAPI** is a great choice to build the CASM service – it's asynchronous, fast, and easy to write and auto-docs the APIs with OpenAPI. It will allow handling multiple agent requests concurrently (important when many agents hit memory at once). FastAPI also makes it straightforward to add WebSocket endpoints for real-time updates if using that. If you go with Go, using a minimal framework like **Gin** or **Fiber** can get you up and running quickly with HTTP handlers. Ensure the framework supports whatever auth method you implement (e.g. middleware for API key checking). Also, consider using **Swagger/OpenAPI** to describe your endpoints; both FastAPI and many Go frameworks can auto-generate docs which is useful for early adoption by others.
- **DevOps and Cloud:** In early stages, simplicity is key. You can deploy CASM on a single VM or container in cloud (AWS, GCP, Azure, or even Heroku-like platforms). Containerization with Docker is recommended – Docker Compose can orchestrate the service plus a Postgres and Neo4j container for local dev and testing ⁴¹. For production, you might use a managed Postgres service (for reliability) and run your CASM service in a container on a service like AWS ECS/Fargate or Azure Container Apps. Using **Kubernetes** would be overkill at startup MVP stage, but plan for an easy path to scale out: e.g. avoid local state in the service so you can run multiple replicas behind a load balancer when needed. For logging/monitoring, using cloud services or simple setups like pushing logs to CloudWatch/Stackdriver is fine. Set up backups for your database (most managed DBs have this out-of-box).

In summary, a **proven configuration** is: *Python + FastAPI service, Postgres (with pgvector) for main memory storage, sentence-transformers for embeddings, and possibly Neo4j for graph if needed* ¹⁶. This aligns with what Eion (an open-source shared memory system) uses, thus you'd be in good company following that blueprint ¹⁷. Over time, you can modularize and replace components: for instance, swap Postgres out for a distributed DB or add Redis cache for faster reads, or offload vector search to a specialized service when query load grows ⁴². The stack outlined gives you a balance of **rapid development** (leveraging Python's AI ecosystem) and **scalability options** (robust databases, modular components) which is ideal for a startup-grade infrastructure product.

5. Cost Estimate and Team Size

Building and running a CASM system involves both development effort and ongoing infrastructure costs. Below is an estimate of the costs for an early-stage deployment (supporting ~10 agents and modest data volumes), along with considerations for scaling:

- **Development Cost:** To implement the MVP in ~3 months, you'd likely need a small engineering team. In early startup mode this might be 1–2 full-time engineers (perhaps one focusing on backend/API and another on ML/embeddings and data layer). Assuming market rates, two engineers over 3 months could cost on the order of \$40k–\$60k in salary (or opportunity cost). If you're a founder-led team coding it yourselves, the direct monetary cost is lower, but this timeline effectively represents ~ man-months of effort. Budget also for ancillary needs like project management or consulting (maybe \$5k–\$10k if you need expert advice on CRDTs or security). The development cost is largely upfront engineering time – using open-source

components (databases, libraries) means you're not paying licensing fees for the software itself at this stage.

- **Hosting & Compute:** For an MVP serving 10+ agents, the load will be low (likely a few requests per second at most, and a small memory store). You can run the entire system on a single cloud VM during development. For example, a \$40-\$80 per month cloud instance (2-4 CPU, 8GB RAM range) could host the CASM service, Postgres, and even a vector index if needed. In the very early phase, you might even use free tiers (e.g. free tier of Heroku or Render for the API, free tier of Neon or Supabase for Postgres) to minimize cost. However, for a more stable deployment, budget around **\$100-\$200/month** for a decent VM or two (one for app, one for DB). If using managed services: a small managed Postgres instance might be \$50/month, a managed vector DB like Pinecone has usage-based pricing (for light usage maybe under \$100/month). A Neo4j AuraDB instance (graph DB cloud) small tier is also in the tens of dollars per month. In summary, for initial hosting, **\$200-\$500/month** should cover a basic setup with some redundancy (e.g. a primary DB and a backup server). Make sure to include cost of data egress if your agents are in different cloud regions from CASM, though for 10 agents text data, this is negligible.
- **Storage Costs:** Memory data itself (text and embeddings) doesn't consume huge space at small scale. For instance, 10 agents over a few weeks might store a few thousand memory items. Each item might be a few hundred bytes of text plus a vector (a 384-dim float vector is ~1.5KB). So we're talking only a few MBs of data initially. Even at larger scale, say 100k memories, that could be on the order of a few hundred MBs including index overhead. This will easily fit in a small Postgres (which often comes with many GBs of storage by default). So storage cost is minimal at first – essentially rolled into your DB instance cost. Just be mindful that if you store every interaction and do not prune, it can grow; but even 1 million short texts is on the order of single-digit GBs. It's good practice to budget for backups (e.g. storing snapshots in S3 or similar) – but early on, manual backups or snapshots are fine and cheap (pennies on cloud storage for a few GBs). Thus, **storage cost** in the first year is likely <\$10/month. It may increase if you accumulate a long history or store large file embeddings, but those decisions can be managed (like compressing or summarizing older memory).
- **Compute for Embeddings & Processing:** If you use external APIs (OpenAI, etc.) for embeddings or other LLM calls as part of CASM, that introduces usage-based costs. For example, OpenAI's embedding is \$0.0004 per 1K tokens – if you embed 1000 pieces of text averaging 100 tokens each, that's \$0.04. This is negligible at small scale, but keep an eye if you plan to embed extremely large volumes of data. Running your own embedding model (like MiniLM) on CPU is essentially covered by the VM cost (slower but no extra cost). If you want faster embedding generation, you might use a GPU cloud instance for that – but for an MVP with low throughput, it's not necessary. Summarization or other LLM usage could be more expensive (e.g. generating long summaries might cost a few cents each on GPT-4). However, since CASM's primary function is storage/retrieval, you can choose to do heavy LLM processing offline or client-side to minimize CASM's direct compute costs.
- **Scaling Projection:** As you scale to more agents or more data, costs will rise. For example, supporting 100+ agents concurrently might require multiple app server instances (each ~\$50-100/mo) behind a load balancer, and a more robust database cluster. A production-ready distributed setup (with high availability Postgres, read replicas, monitoring, etc.) could run in the low thousands per month when you include all pieces. Vector DB usage costs would scale with number of vectors and query rate (for Pinecone, for instance, a pod that stores up to 1M vectors and sustains multiple queries per second can be a few hundred \$/month). But these costs only come into play once you have significant usage. Early on, a single-node approach keeps it to a

few hundred dollars monthly. You should plan that by the time you have paying users or >10 agent processes, you might invest maybe **\\$1k-\\$2k/month** on infrastructure to ensure reliability (redundant servers, better hardware, monitoring tools, etc.).

- **Team size (early stage):** In the first 6–12 months, an **engineering team of 2–4 people** can develop and maintain CASM. This typically includes 1–2 backend engineers (building the core service and APIs), 1 engineer focusing on the ML/AI side (embedding models, any advanced memory algorithms), and maybe a part-time devops or a backend engineer doubling in that role (to manage cloud deployments, CI/CD, scaling). If the startup is larger, a product manager or tech lead might coordinate integration with agent frameworks, but purely for building the infrastructure, a small focused team is sufficient. Each additional feature (like a web console UI, or complex analytics) might justify adding another specialist. But until you scale user adoption, a lean team can handle development. For maintenance (once live), you'd want on-call rotation between a couple of engineers for uptime. Keep in mind, as memory systems are critical, you want someone who can respond to incidents (crashes, data issues) even if team is small, so train multiple people on the system.

Summing up, the initial **capital expense is mainly engineering time**, while the **operational expense is a few hundred dollars per month** for cloud resources. This makes CASM quite accessible to build as a startup product. Just avoid over-provisioning early (you can always scale up as agent load increases) and leverage open-source and cloud credits if available to keep costs low. *(Note: These estimates are based on current cloud pricing and known usage patterns; exact costs can vary. We did not find a single source with all these numbers, so we extrapolated from common setups and publicly available pricing.)*

6. Maintenance Plan

Once CASM is up and running, a solid maintenance and operations plan is essential to ensure data integrity, high availability, and smooth performance. Here are the key aspects of maintaining a CASM system and how to address them:

- **Data Integrity and Consistency:** Protecting the accuracy of shared memory is top priority. Use database constraints and validation to prevent bad data writes – for example, if memory schema expects certain fields, enforce those at the API layer (as discussed with JSON schema validation ¹⁰). Regularly run integrity checks: e.g., verify vector index entries match memory records count, or that knowledge graph links refer to valid nodes. Implement unit tests for any transformation logic (like summarizations or merges) to ensure they don't corrupt data. Additionally, consider periodic **memory audits**: an automated job or a maintenance agent that reviews recent memory changes for anomalies (e.g. an agent that wrote an oddly large number of entries or contradictory facts) and flags them. In later stages, incorporating an *audit trail* where every change is logged with who/when (as we stored provenance) allows manual review if something seems off ⁴³. If an inconsistency is found (say an agent mistakenly writes an incorrect fact), having version history makes it possible to revert or correct it. Thus, keep historical records or at least backups of the database state to restore if needed.
- **Updates and Schema Changes:** As the system evolves, you'll update schemas or add new memory types. Use migrations for database changes (tools like Alembic for SQL migrations if using Python, or Liquibase, etc.). Maintain backward compatibility in APIs when possible – if you introduce a new field, make it optional so older agents don't break. Communicate changes to any agent developers using CASM. For the service itself, adopt a versioning strategy (e.g. version your API endpoints like `/v1/memory` so that future breaking changes can go under `/v2`). This

way, if you improve how memory is stored or indexed, you can run both versions for a transition period. Always test migrations on staging data to ensure no data loss. Given agents might be using the memory continuously, plan for **zero-downtime deployments**: e.g., use rolling deployments or toggles so that updates don't require full downtime. If the system must be taken offline for a big change (should be rare for an infra product), schedule it in an off-peak time and notify stakeholders.

- **High Availability and Failover:** In production, CASM should be resilient to failures since it could become a central brain for many agents. This means eliminating single points of failure. For the database, use replication: a primary and at least one replica that can take over if primary fails. Many managed DB services support failover automatically. For the application service, run multiple instances in different availability zones behind a load balancer – so if one machine or zone goes down, others still serve agents. Implement health checks: the CASM service should expose a `/health` endpoint (like Eion's example ⁴⁴) so that the load balancer or orchestrator can detect a failed instance and reroute traffic. If using Redis or similar in-memory components, consider using their clustering or master-slave modes with automatic failover. **Failover procedures** need to be documented: e.g., if the primary DB fails, how do you promote a replica? Automate this if possible or use managed services that handle it. Regularly simulate failure scenarios (kill a node, see if system stays up) to be confident in the HA setup.
- **Backups and Recovery:** Even with redundancy, backups are your safety net against data corruption or catastrophic failures. Schedule automated daily or hourly backups of the core data store (for Postgres, enable point-in-time recovery backups if using a managed service, or run `pg_dump` nightly for offsite backups). Similarly, if using Neo4j or other stores, back them up on a schedule. Store backups in a secure, redundant location (e.g., cloud storage bucket). Test the restore process occasionally – a backup is only good if it can be restored correctly. This means trying to restore on a staging instance to verify everything comes back intact. Also, implement logical backups/export for the knowledge graph if it's separate (Neo4j has tools for this). In addition, if using a vector DB that doesn't easily dump data, keep a copy of your vectors or be able to re-compute them from text in worst case. For the application itself, keep copies of config files (like schema definitions, ACL config ²³) under version control or backed up so you can rebuild the environment.
- **Performance Monitoring and Scaling:** Use monitoring tools to keep an eye on CASM's performance metrics. Key things to track: database CPU/IO load (to catch if queries are slow), memory usage (to ensure cache or DB isn't running out of RAM), API response times, and throughput (requests per second). Also track vector search performance (if queries start taking longer as data grows, it might be time to switch to approximate search or scale out). Services like DataDog, NewRelic, or open-source Prometheus+Grafana can be integrated to gather metrics. At minimum, enable logging of slow queries on the database and instrument the code to log any API call taking >X ms. This will help pinpoint bottlenecks. As usage grows, be ready to scale vertically (bigger instance) or horizontally (more instances or read-replicas). For example, if semantic searches become heavy, you might spin up a separate vector search service or use a dedicated vector DB cluster at that time. **Capacity planning** is part of maintenance: define thresholds when you need to add more resources (e.g. when CPU consistently > 70% or when memory usage > 80%, scale up). Because this is infrastructure for others, you should ideally **over-provision slightly** than under-provision – a small buffer ensures smooth performance under spikes.
- **Logging and Usage Monitoring:** Maintain detailed logs of operations. Logs are not just for debugging, but can feed into understanding usage patterns. For instance, monitor how many

memory entries are being created per day, or which agents call the API most. This helps in both optimizing (you might find a certain agent spams the search API and decide to add rate limiting or caching for it) and in showcasing the value of CASM (e.g. “our memory system served X queries without failure this week” is a good metric). Consider building a simple dashboard or using existing analytics: the CASM console (if you build one) can show active sessions, number of agents, memory size, etc. ⁴⁵. Eion’s console has a monitoring tab for agent status ⁴⁵ – you could implement a pared-down version that at least lists connected agents and recent ops. For anomaly detection, set up alerts: e.g., if error rate goes above some threshold or if response time spikes, page an on-call engineer. This can often be done with cloud monitor services or a tool like Sentry for error tracking.

- **Regular Updates and Patches:** Keep your stack updated for security and stability. Apply patches to the database and dependencies (e.g. if a new version of Postgres or Neo4j fixes a bug or security hole, schedule an upgrade). The same applies to libraries like FastAPI, and to the server OS if you manage your own VM. Automating security updates or using managed services reduces this burden. Also update the embedding model periodically if better versions become available (ensuring to re-index if needed). Each update should be tested in a staging environment before production rollout to avoid surprises.
- **User Support and Issue Resolution:** As a startup product, you might have external users (or internal teams) relying on CASM. Provide a channel for them to report issues (Slack or email). When a bug or performance issue is reported, have a process to triage and fix it promptly, as memory issues can block the entire multi-agent workflow. Keeping the system transparent can help here – e.g., if agents get an error from CASM, ensure the error messages are informative. Having good documentation reduces maintenance load too (fewer support questions if things are clearly explained).

In essence, maintaining CASM is about being **proactive** – regularly checking system health, safeguarding data with backups and audits, and scaling ahead of demand. Because CASM becomes critical infrastructure for AI agents, investing in reliability from day one (even if it’s just scripts for backups and basic monitors) pays off by preventing downtime and data loss. While our maintenance plan advice is drawn from standard best practices (with references to the design choices from Eion and others), it’s tailored for an AI memory context where consistency and availability are paramount.

7. Future Opportunities and Advancements

Once the core CASM system is in place, there are several avenues to extend its capabilities. These enhancements can differentiate your product and provide more value to users (engineers and researchers using CASM). Here are some promising opportunities for future development:

- **Deeper Contextual Awareness:** Currently, CASM retrieves memories largely by semantic similarity or direct keys. Future versions can incorporate more context awareness to deliver the *right* information at the right time ⁴⁶. For example, integrate **contextual filters** that automatically consider the querying agent’s role or the current task stage. An advanced CASM could detect an agent’s context (from its prompt or situation) and proactively push relevant memory snippets to it. Another aspect is **temporal context** – using the time dimension not just for versioning but for relevance (e.g., recent memories might be boosted in ranking, or CASM could automatically decay old irrelevant info ¹¹ ¹²). Contextual awareness could also mean environment context: if agents operate in a physical or simulation environment, CASM could store state about that environment (shared world memory) that agents use. An example future

feature: *context-conditioned queries*, where an agent asks CASM a question and CASM uses not only the question text but also the agent's recent behavior or profile to answer (essentially a query expansion). This moves CASM closer to an intelligent memory that adapts to situation, rather than a static database.

- **Memory Audit Trails and Explanations:** As mentioned under maintenance, CASM can log all changes with who and when. Exposing this as a feature creates **memory audit trails** that are valuable in enterprise settings. Users could query “who added this piece of knowledge and why?”. Building an interface (or API) to browse the history of a memory – showing its evolution over time – makes the system more transparent ⁴³. This is useful for debugging agent behaviors (trace which memory led an agent to a conclusion) and for compliance (in domains like finance or healthcare, being able to audit what info an AI used is crucial). In the future, CASM might even integrate an *explainability module*: e.g., provide citations or source tracking for knowledge in memory. If an agent cites a document, the memory could carry a reference link. This way, when another agent uses that knowledge, CASM can supply the provenance (e.g., “This fact comes from Report X, added by AnalystAgent yesterday”). Such features turn CASM into not just storage but a **knowledge provenance system**, increasing trust in multi-agent outcomes.
- **Multi-User and Multi-Tenant Support:** Right now, we consider multiple agents possibly all within one team or application. In the future, CASM could support **multiple distinct user groups or tenants**, each with isolated (or partially shared) memory. For example, if offered as a service, different companies might use one CASM but keep their data separate. This requires robust **access control** – possibly extending the simple ACLs to a full permission system with roles, groups, and maybe encryption for certain data. You could allow some memory to be globally shared while other memory is private to a subset of agents or users. This ties in with the concept of *federation* (below) as well. Another angle is hierarchical access: e.g., an admin user can see and edit all memory, while a normal agent can only append but not delete, etc. In crew scenarios, maybe each “crew” (group of agents working for a specific project) has its own memory space. The groundwork for this is laid by the agent identity and permission model in current design ²⁵ ²³, but it can be extended. In terms of implementation, you might incorporate an identity service (to manage users/humans who oversee agents) and tie that to memory spaces. Technically, adding a `tenant_id` or `team_id` field to memory entries and enforcing queries filter by those would achieve basic multi-tenancy. Down the line, offering CASM as a multi-tenant SaaS product could open new business, where each client gets a logically separate memory space, securely partitioned.
- **Federated Memory and Edge Deployment:** As CASM scales, there may be cases where a central repository is not ideal (due to latency or privacy). **Federated memory** would allow multiple CASM instances to sync or share knowledge without a single centralized DB. In a federated setup, each agent (or agent cluster) might have a local memory store that occasionally merges with others. This is analogous to federated learning but for knowledge: each node only shares certain summaries or updates, preserving sensitive data locally. To implement this, CRDTs become even more crucial – they would allow memory updates to be exchanged between CASM nodes and merged without conflict ⁹. Another scenario is edge computing: imagine personal AI assistants each running CASM on a user's device, but with a global CASM cloud that aggregates common knowledge. Federated design would let them share high-level knowledge (or queries) without uploading everything (helping with privacy). This is a forward-looking feature; to make it happen, you'd need to package CASM in a lightweight form (maybe as a library or local service) and develop a sync protocol. Perhaps using something like client-side databases (SQLite with CRDT, or OrbitDB in browser context) and a central aggregator. While

complex, this could differentiate CASM by addressing data residency concerns and reducing latency for geographically distributed agents.

- **Cognitive Enhancements (Memory Compression, Chunking):** Future CASM versions could incorporate more AI to manage memory itself. One idea is **automatic memory compression**: as memory grows, CASM could periodically summarize or distill less-used memories (similar to the “distillation” approach described for conversational memory ⁴⁷ ⁴⁸). This would be an automated background process that ensures the memory store remains high-signal and not cluttered with redundant or outdated info. Another enhancement is **memory chunking or grouping** – CASM could organize memories into contexts or themes. For instance, it could detect that certain facts frequently are used together and create a meta-memory (like a sub-graph or an “index of related items”). This begins to blur into LlamaIndex territory (which builds indices for better retrieval), but doing it internally could make retrieval smarter. These features would leverage LLMs or ML algorithms running within CASM: e.g., use an LLM to read a cluster of memory and produce a concise summary that agents can use instead of raw data.
- **Integration of Learning Systems:** In the long run, CASM might not just store knowledge, but also help *learn* from it. For example, using the accumulated memory to fine-tune models or train specialized classifiers. A concrete idea: CASM keeps track of which memories were useful (perhaps agents give feedback or CASM observes that certain retrieved info led to successful outcomes). CASM could then score memory entries by utility. These signals could train a model to predict what knowledge is most relevant for a given situation, improving retrieval beyond pure semantic similarity. Another aspect is using memory for **behavior adaptation** – e.g., if CASM logs show an agent often contradicts another on a certain topic, the system could alert or adjust context to mitigate that. This moves CASM into a more active coordinator role, maybe overlapping with orchestration frameworks but from a memory perspective.

In pursuing these advanced features, it's wise to involve your user base (the engineers and teams using CASM) to see what they need most. For example, enterprise users might strongly value the audit trail and access control features, whereas academic users might be more excited about federation and open collaboration. Each opportunity (context-awareness, auditability, multi-tenancy, federation, intelligent memory management) adds complexity, so they would likely be staged over multiple versions of the product. But they all align with the vision of CASM being a **powerful memory infrastructure** that keeps improving agents' collective intelligence and reliability.

To sum up: CASM's roadmap can evolve from a basic “data store” to an **intelligent memory platform** – one that not only stores info, but understands context, explains itself, spans across boundaries, and continuously learns what knowledge matters. This will cement CASM's position in the AI agent ecosystem as an indispensable component for sophisticated multi-agent applications.

8. Comparison with Existing Tools & Frameworks

There are several existing tools and frameworks that address parts of the multi-agent memory challenge. It's important to understand how CASM compares and differs from each, to position our system and learn from their approaches:

- **LangChain Memory:** LangChain provides memory abstractions for LLM applications, such as `ConversationBufferMemory` (which keeps a buffer of recent chat messages) and even a `KnowledgeGraphMemory` that can integrate with vector stores ⁴² ⁴⁹. These are **developer-friendly utilities** within a single-agent (or single-session) context. For example, a

ConversationBufferMemory simply stores the chat history in memory or in a database, and feeds it into the prompt each turn ⁵⁰. LangChain's memory components can indeed use external stores (e.g. Pinecone or a database) to persist information between sessions, so they do offer persistence. However, LangChain's focus is on *embedding memory into the agent workflow* rather than a standalone memory service. By default, memories are typically isolated per agent or per chain – LangChain doesn't inherently provide a multi-agent shared memory space (though one could hack it by pointing multiple agents to the same memory object). Also, **LangChain lacks built-in conflict resolution or concurrent update handling** because it assumes a more sequential use (one agent appending to its history). It's primarily an in-process solution; if you had two separate agent processes, they'd have to coordinate outside of LangChain's provided tools. In contrast, CASM is a separate infrastructure service designed for **multiple agents concurrently**, with stronger guarantees on consistency and richer query capabilities (LangChain memory often does simple lookups or vector similarity, but doesn't manage complex relationships or global indexing on its own). One could imagine CASM being used *under* LangChain's hood: indeed, integration is possible (for instance, using CASM's API to implement a custom `ChatMessageHistory` class in LangChain). To summarize: LangChain memory makes it easy to give an agent memory, but CASM aims to be a **universal, shareable memory** across agents, with enterprise-grade features (versioning, multi-agent sync) that LangChain doesn't natively have.

- **AutoGen (Microsoft):** AutoGen is an open-source framework by Microsoft for orchestrating multi-agent LLM systems through conversational paradigms ⁵¹. It excels at agent-agent dialogue management, allowing you to set up multiple agents that communicate by exchanging messages. AutoGen includes some memory handling – each agent maintains a message history (like a chat transcript) by default ⁵². For more persistent memory, AutoGen can integrate with external data stores or user-defined memory modules. However, **AutoGen does not come with a dedicated cross-agent memory store out-of-the-box**. Agents primarily share information by messaging each other or by reading/writing from external tools/resources as configured. In fact, the community around AutoGen has identified this gap: projects like *memX* were explicitly created to give AutoGen (and similar frameworks) a shared memory layer ⁵³. MemX's README notes that without a tool like it, multi-agent setups (Autogen, LangChain's LangGraph, etc.) “lack a reliable way to share evolving context” and thus rely on passing messages or having a central controller ⁵³. So CASM is addressing the same pain point memX did – providing that missing shared state. Compared to rolling your own with AutoGen, CASM offers a pre-built solution where memory is first-class. AutoGen users could call CASM's APIs as tools within the AutoGen conversation (similar to how one might call a database). The benefit is CASM would handle merging and indexing beyond just what AutoGen's simple history can do. In short, AutoGen is great for orchestration logic and conversation handling, while CASM would complement it by handling **persistent, shared state** that persists across the entire multi-agent workflow. As CASM matures, integration adapters for frameworks like AutoGen would be useful (e.g., a wrapper that lets AutoGen agents call CASM easily), making the two synergistic rather than competing.
- **CrewAI:** CrewAI is an open-source multi-agent orchestration framework that focuses on creating “crews” of agents with roles, tools, and planning ⁵⁴ ⁵⁵. It specifically emphasizes making it easy to define agents and tasks (even via no-code interfaces) and have them collaborate. Regarding memory, CrewAI has recognized the importance of agent memory – their documentation mentions a “sophisticated memory-management system” giving agents **shared short-term, long-term, entity, and contextual memory** ⁵⁶. This implies that CrewAI provides some built-in memory constructs. Likely, short-term memory is akin to recent chat history, long-term might interface with a vector store, entity memory might be a knowledge base of facts, and contextual memory perhaps relates to the current task context. CrewAI probably uses vector

databases and other storage under the hood (the mention of integration with Weaviate for cross-agent knowledge sharing ⁵⁷ suggests they hook into vector DBs or other stores). The difference is CrewAI's memory is a *feature within a larger framework*, not a general-purpose memory service. If you use CrewAI, you get a lot of functionality (planning, tools, etc.) and memory is handled in that ecosystem. But if you wanted to use that memory outside CrewAI, or use it with a different agent system, it's not straightforward. CASM, on the other hand, is **framework-agnostic** – it doesn't orchestrate agents, it serves any agent. One might compare CASM to CrewAI's memory and find they aim for similar end goals (a shared knowledge base for agents), but CASM could be used *with or without* CrewAI. In fact, one could integrate CASM as the backend for CrewAI's memory if CrewAI allows (CrewAI being open-source might allow swapping out memory backend). Also, CASM's design (with CRDTs, versioning) might be more advanced in conflict handling than CrewAI's current memory (which, if memory isolation issues on community forums are to note, might have had some "flakiness" or limitations). CrewAI's strength is ease of orchestration; CASM's strength is robust, scalable memory – they can complement. If an organization already uses CrewAI but finds its memory insufficient (e.g., not scaling or not persistent enough), CASM could serve as an external memory store that CrewAI agents use for any heavy-duty knowledge sharing.

- **LlamaIndex (GPT Index):** LlamaIndex is a framework for connecting LLMs to external data through indices (vector indices, keyword tables, knowledge graphs, etc.). It's often used to enable retrieval of documents or facts for question-answering tasks. In context of agent memory, LlamaIndex can be repurposed to *store and query agent-acquired knowledge*. For example, one could use LlamaIndex to build a knowledge graph index of all facts an agent accumulates, or a vector index of all past dialogues. LlamaIndex provides tools to create these indices and query them with natural language. It effectively gives you an "augmentation" to memory – but it's not a multi-agent system per se; it's more of an API that a single agent (or a controlling program) uses to fetch info. **LlamaIndex's capabilities overlap with CASM's vector search and knowledge graph components.** In fact, if you were to implement CASM's functionality quickly without building from scratch, you might leverage LlamaIndex under the hood: it can manage a vector store or a graph of knowledge for you. However, LlamaIndex by itself is an SDK, not a server – it runs within your application process. So it doesn't inherently solve multi-agent concurrency (though multiple agents could query the same index if the app is built that way). Also, LlamaIndex's indices are often designed to be rebuilt or updated in batch, rather than handling high-frequency concurrent writes that an active multi-agent system might generate. CASM is designed as a **live database** that can handle continuous updates and queries. LlamaIndex is more like a library to facilitate building a memory solution. As one source notes, LlamaIndex can provide long-term memory and context-aware retrieval so agents can use large corpora in real-time ⁵⁸ – this means if you have a document store, LlamaIndex helps query it to give context to agents. CASM could actually store that document corpus and provide an API; LlamaIndex could be one way an agent queries CASM (by treating CASM's data as the knowledge corpus). Essentially, LlamaIndex is a higher-level toolkit, whereas CASM is a lower-level infrastructure. They can be seen as complementary: CASM manages the data storage and consistency, LlamaIndex (on the client side) could manage how to query and combine data for a prompt. If we compare directly: CASM aims to be **persistent and multi-agent**, LlamaIndex aims to be **convenient and flexible for building data augmented prompts**. One advantage CASM would have as a product is that it's centrally hosted and multi-client, whereas LlamaIndex you'd have to integrate into each project separately.

- **Other related tools:** The question specifically asked about the above, but it's worth mentioning in passing: **LangChain's LangGraph** (sometimes referred to in community) which tries to create agent workflows with graph structure – memory is still typically via vector stores or context

passing, so CASM could plug in there. **MemGPT** (Memory GPT) is mentioned in Eion's context as a persistent storage wrapper around GPT; that indicates approaches like compressing and storing conversation chunks. These are more experimental and not general infrastructure. **Zep** is an open-source LLM memory server (by Timber) that provides long-term conversation memory (with vector search and etc.), quite similar in spirit to CASM. Zep wasn't explicitly asked, but it's another competitor in this space. CASM's unique selling point would be the combination of features (CRDT-driven consistency, knowledge graphs, integrated vector search, multi-agent focus) which individually exist in pieces in various tools but not collectively in one. For instance, CrewAI has shared memory but not as an independent product; Zep has vector memory for chat but not knowledge graphs; LlamaIndex has graphs but not a server/API; etc. CASM as envisioned here tries to be the **"Google Docs" for AI agents** ⁷ – a real-time collaborative memory layer with robust infrastructure qualities, which currently only early projects like Eion are aiming for.

Summary of Comparison: LangChain and LlamaIndex are more developer libraries for memory and retrieval within an app; AutoGen and CrewAI are orchestration frameworks that acknowledge memory but don't fully solve shared persistent memory; none of these provide the full suite of features (distributed consistency, rich schemas, multi-agent concurrency handling) that CASM targets. CASM is meant as a standalone, **platform-level solution** – something that a variety of agent frameworks or custom agents can all plug into as clients. By focusing on being framework-agnostic and scalable, CASM differentiates itself from these tools that are either tied to their ecosystem (LangChain, CrewAI) or not built for multi-agent collaboration out-of-the-box (AutoGen, LlamaIndex). We see CASM not as reinventing the wheel but rather combining the best of these approaches: like LangChain's easy integration of vector stores ⁴², AutoGen's multi-agent dialog, CrewAI's shared context concept ⁵⁶, and LlamaIndex's knowledge organization – into a **cohesive infrastructure product**.

Ultimately, CASM can coexist with these tools, enhancing them: for example, a CrewAI or AutoGen user can incorporate CASM to handle a complex knowledge base that persists beyond a single run, or a LangChain pipeline can swap its ephemeral memory with CASM calls for persistence and sharing. By understanding the landscape, we ensure CASM is compatible and complementary, while offering capabilities (like conflict-free syncing and comprehensive memory graph management) that set it apart from current solutions.

Sources:

1. Kim, Min. "Eion: The 'Google Docs' for AI Agents – Solving the Shared Memory Problem." Medium, Jul 2025 ⁶ ⁷ ¹⁶ ¹⁷.
2. Eion README – Shared Memory Storage for Multi-Agent Systems (GitHub eiondb/eion) ⁵⁹ ¹⁸ ¹⁵ ²⁶.
3. Liles, Doug. "Understanding the Role of Shared Memory Graphs in Multi-Agent Systems." aiagenthive.org, Oct 2025 ² ³ ⁶⁰.
4. Ghosh, Bijit. "Agents That Remember: Temporal Knowledge Graphs as Long-Term Memory." Medium, 2025 ¹³ ⁴³ ¹¹ ²¹.
5. Cauri. "Memory in multi-agent systems: technical implementations." Medium, May 2025 ¹⁹ ⁴⁷ ⁴⁸.
6. **MemX README** – Mehul Gupta, *memX: Real-time shared memory layer for LLM agents* (GitHub MehulG/memX) ¹⁰ ²⁰ ⁵³ ²³.
7. Bazeley, Mikiko. "Why Multi-Agent Systems Need Memory Engineering." MongoDB Blog on Medium, Sep 2025 ¹ ³⁹.
8. InfoQ Article – Kumar, Roshan. "Merge Replication vs CRDT for Multi-Master Databases." InfoQ, 2018 ⁸ ⁹.
9. CrewAI Documentation – *CrewAI open-source framework overview*, crewai.com, 2023 ⁵⁶.
10. Sparkco AI Blog – "Deep Dive into Knowledge Graph Memory Innovations." Oct 2025 ⁴² ⁴⁹.

11. ScrapeGraphAI Blog – “Advanced Multi-Agent Systems: LangGraph, LlamaIndex, CrewAI” – FAQ section ⁵⁸ .
12. GitHub – Eion Demo (animated).
13. Eion Quickstart (GitHub) ²⁹ ³⁶ ⁴⁴ .
14. **Additional references:** AutoGen framework description ⁵¹ , CrewAI integration with Weaviate ⁵⁷ , LangChain memory code examples ⁵⁰ .

¹ ³⁹ ⁴⁰ ⁴⁶ Why Multi-Agent Systems Need Memory Engineering | MongoDB
<https://medium.com/mongodb/why-multi-agent-systems-need-memory-engineering-153a81f8d5be>

² ³ ⁶⁰ Understanding the Role of Shared Memory Graphs in Multi-Agent Systems
<https://aiagenthive.org/blog/understanding-the-role-of-shared-memory-graphs-in-multi-agent-systems>

⁴ ⁵ ⁶ ⁷ ¹⁶ ¹⁷ ²⁸ Eion: The “Google Docs” for AI Agents–Solving the Shared Memory Problem | by Min Kim | Medium
<https://medium.com/@mingyouk/eion-the-google-docs-for-ai-agents-solving-the-shared-memory-problem-adaf946321f6>

⁸ ⁹ Picking an Active-Active Geo Distribution Strategy: Comparing Merge Replication and CRDT - InfoQ
<https://www.infoq.com/articles/database-merge-replication-crdt/>

¹⁰ ²⁰ ²³ ²⁴ ³⁴ ⁵³ GitHub - MehulG/memX: A real-time shared memory layer for multi-agent LLM systems.
<https://github.com/MehulG/memX>

¹¹ ¹² ¹³ ²¹ ²² ⁴³ Agents That Remember, Temporal Knowledge Graphs as Long-Term Memory | by Bijit Ghosh | Medium
<https://medium.com/@bijit211987/agents-that-remember-temporal-knowledge-graphs-as-long-term-memory-2405377f4d51>

¹⁴ ¹⁵ ¹⁸ ²⁵ ²⁶ ²⁷ ²⁹ ³² ³³ ³⁵ ³⁶ ⁴¹ ⁴⁴ ⁴⁵ ⁵⁹ GitHub - eiondb/eion: Shared Memory Storage for Multi-Agent Systems
<https://github.com/eiondb/eion>

¹⁹ ³⁰ ³¹ ³⁷ ³⁸ ⁴⁷ ⁴⁸ Memory in multi-agent systems: technical implementations | by cauri | Medium
<https://medium.com/@cauri/memory-in-multi-agent-systems-technical-implementations-770494c0eca7>

⁴² ⁴⁹ ⁵⁰ Deep Dive into Knowledge Graph Memory Innovations
<https://sparkco.ai/blog/deep-dive-into-knowledge-graph-memory-innovations>

⁵¹ AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent ...
<https://www.microsoft.com/en-us/research/publication/autogen-enabling-next-gen-llm-applications-via-multi-agent-conversation-framework/>

⁵² Microsoft AutoGen: Orchestrating Multi-Agent LLM Systems | Tribe AI
<https://www.tribe.ai/applied-ai/microsoft-autogen-orchestrating-multi-agent-llm-systems>

⁵⁴ ⁵⁵ ⁵⁶ Opensource
<https://www.crewai.com/opensource>

⁵⁷ Building Multi-Agent Systems with Crew AI and Weaviate
<https://weaviate.io/blog/building-multi-agent-systems>

⁵⁸ Advanced Multi-Agent Systems: Integrating LangGraph, LlamaIndex, and CrewAI | ScrapeGraphAI
<https://scrapegraphai.com/blog/multi-agent/>